

### 0. Quick Essentials

```
#include <bits/stdc++.h> // includes everything - use in CP
using namespace std;

// Limits
INT_MAX, INT_MIN // 2^31 - 1, -2^31
LLONG_MAX, LLONG_MIN // long long limits
1e9 + 7 // common MOD
numeric_limits<int>::max() // portable MAX

// Type sizes
int // 32-bit, ~2e9
long long // 64-bit, ~9.2e18
__int128 // 128-bit (no cin/cout - use custom print)
double // 64-bit float
bool, char, short

// Useful macros (CP style)
#define ll long long
#define pii pair<int,int>
#define vi vector<int>
#define pb push_back
#define mp make_pair
#define all(v) v.begin(), v.end()
#define sz(v) (int)v.size()
```

### 1. Array

```
// Static array
int arr[100];
int arr[5] = {1, 2, 3, 4, 5};
int grid[100][100]; // 2D

// STL array (fixed size, stack-allocated)
#include <array>
array<int, 5> a = {1, 2, 3, 4, 5};
a[i]; // 0(1) access
a.size();
a.fill(0); // fill with value
a.front(), a.back();

// Algorithms on arrays
sort(arr, arr + n);
reverse(arr, arr + n);
fill(arr, arr + n, 0);
memset(arr, 0, sizeof(arr)); // only use with 0 or -1
```

### 2. Vector

```
#include <vector>
```

```

vector<int> v; // empty
vector<int> v(n, 0); // size n, filled with 0
vector<int> v = {1, 2, 3};
vector<vector<int>> grid(n, vector<int>(m, 0)); // 2D grid

// Operations
v.push_back(x); // O(1) amortized
v.pop_back(); // O(1)
v.insert(v.begin()+i, x); // O(n)
v.erase(v.begin()+i); // O(n)
v.erase(v.begin()+l, v.begin()+r); // range erase
v[i]; // O(1) access (no bounds check)
v.at(i); // O(1) access (with bounds check)
v.front(), v.back(); // first, last element
v.size(); // number of elements
v.empty(); // true if size == 0
v.clear(); // remove all elements
v.resize(n); // resize
v.reserve(n); // preallocate capacity (avoids reallocation)

// Iteration
for (int x : v) { } // range-based
for (int i = 0; i < v.size(); i++) { }
for (auto it = v.begin(); it != v.end(); it++) { }

// Sorting & searching
sort(v.begin(), v.end()); // ascending O(n log n)
sort(v.begin(), v.end(), greater<int>()); // descending
sort(v.begin(), v.end(), [](int a, int b){ return a > b; }); // custom
reverse(v.begin(), v.end());
auto it = find(v.begin(), v.end(), x); // O(n) linear search
int idx = it - v.begin(); // convert iterator to index

// Useful
int mn = *min_element(all(v));
int mx = *max_element(all(v));
int total = accumulate(all(v), 0); // sum
v.assign(n, val); // reset to n copies of val
unique(v.begin(), v.end()); // remove consecutive duplicates (sort first)
v.erase(unique(all(v)), v.end()); // full dedup

// 2D traversal
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        cout << grid[i][j];

```

### 3. String

```

#include <string>

string s = "hello";
s[i]; // char access
s.length(), s.size(); // length
s + t; // concatenation
s.append(t); // append in-place
s.push_back('c'); // append char
s.pop_back(); // remove last char
s.substr(i, len); // substring starting at i, length len
s.find("sub"); // first occurrence, string::npos if not found
s.rfind("sub"); // last occurrence
s.replace(i, len, "new");
s.compare(t); // 0 if equal
s.empty();
s.clear();

// Convert

```

```

stoi(s), stol(s), stoll(s), // string → int/long/long long
stof(s), stod(s);          // string → float/double
to_string(x);              // int/double → string

// Case & checks
toupper(c), tolower(c);    // works on chars
isdigit(c), isalpha(c), isalnum(c), isspace(c);

// Sort / reverse
sort(s.begin(), s.end());
reverse(s.begin(), s.end());

// Stringstream (split / parse)
#include <sstream>
stringstream ss("1 2 3 4");
int x;
while (ss >> x) v.push_back(x);    // parse ints from string

// Split by delimiter
string token;
stringstream ss2(s);
while (getline(ss2, token, ','))    // split by ','
    tokens.push_back(token);

```

## 4. Stack

```

#include <stack>

stack<int> st;
st.push(x);          // O(1)
st.pop();            // O(1) – does NOT return value
st.top();            // peek – O(1)
st.empty();
st.size();

// Monotonic stack pattern (next greater element)
stack<int> st;
vector<int> res(n, -1);
for (int i = 0; i < n; i++) {
    while (!st.empty() && arr[st.top()] < arr[i]) {
        res[st.top()] = arr[i];
        st.pop();
    }
    st.push(i);
}

```

## 5. Queue

```

#include <queue>

queue<int> q;
q.push(x);           // enqueue – O(1)
q.pop();             // dequeue – O(1) (no return)
q.front();           // peek front – O(1)
q.back();            // peek back – O(1)
q.empty();
q.size();

// BFS template
queue<int> q;
vector<bool> visited(n, false);
q.push(start);
visited[start] = true;

```



```

while (!q.empty()) {
    int node = q.front(); q.pop();
    for (int nei : graph[node]) {
        if (!visited[nei]) {
            visited[nei] = true;
            q.push(nei);
        }
    }
}
}

```

## 6. Deque (Double-Ended Queue)

```

#include <deque>

deque<int> dq;
dq.push_back(x);          // O(1)
dq.push_front(x);         // O(1)
dq.pop_back();            // O(1)
dq.pop_front();           // O(1)
dq.front(), dq.back();
dq[i];                    // O(1) random access
dq.size(), dq.empty();

// Sliding window maximum (monotonic deque)
deque<int> dq; // stores indices
for (int i = 0; i < n; i++) {
    while (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
    while (!dq.empty() && arr[dq.back()] <= arr[i]) dq.pop_back();
    dq.push_back(i);
    if (i >= k - 1) cout << arr[dq.front()] << " ";
}

```

## 7. Priority Queue (Heap)

```

#include <queue>

// MAX-HEAP (default)
priority_queue<int> pq;
pq.push(x);          // O(log n)
pq.pop();            // O(log n)
pq.top();            // O(1)
pq.empty(), pq.size();

// MIN-HEAP
priority_queue<int, vector<int>, greater<int>> pq;

// Custom comparator
auto cmp = [](pair<int,int> a, pair<int,int> b) {
    return a.first > b.first; // min-heap by first
};
priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);

// Pair heap (sorts by first, then second)
priority_queue<pair<int,int>> pq;
pq.push({priority, value});
auto [p, v] = pq.top();

// Build from vector - O(n)
priority_queue<int> pq(v.begin(), v.end());

// Dijkstra template
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
vector<int> dist(n, INT_MAX);

```

```

dist[src] = 0;
pq.push({0, src});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;
    for (auto [v, w] : graph[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
        }
    }
}
}

```

## 8. unordered\_map (Hash Map)

```

#include <unordered_map>

unordered_map<int, int> mp;           // average O(1) ops

mp[key] = val;                       // insert / update
mp[key];                             // access (inserts 0 if missing!)
mp.at(key);                          // access (throws if missing)
mp.count(key);                       // 1 if exists, 0 if not
mp.find(key);                        // returns iterator
mp.erase(key);
mp.size(), mp.empty();
mp.clear();

// Safe access
if (mp.count(key)) val = mp[key];
auto it = mp.find(key);
if (it != mp.end()) val = it->second;

// Iteration
for (auto& [k, v] : mp) cout << k << " " << v;

// Frequency count
for (int x : arr) mp[x]++;

// Custom hash (to avoid anti-hash tests in CP)
struct custom_hash {
    size_t operator()(int x) const {
        x = ((x >> 16) ^ x) * 0x45d9f3b;
        x = ((x >> 16) ^ x) * 0x45d9f3b;
        return (x >> 16) ^ x;
    }
};

unordered_map<int, int, custom_hash> safe_map;

```

## 9. map (Ordered Map / BST)

```

#include <map>

map<int, int> mp;                    // O(log n) ops, sorted by key

mp[key] = val;
mp.at(key);
mp.count(key);
mp.find(key);
mp.erase(key);
mp.size(), mp.empty();

// Range / order queries (not available in unordered_map)

```

```

mp.lower_bound(x);           // first key  $\geq$  x
mp.upper_bound(x);          // first key > x
mp.begin();                 // min key
prev(mp.end());             // max key
mp.rbegin();                // max key (reverse iterator)

// Iteration (always sorted by key)
for (auto& [k, v] : mp) { }

// multimap – allows duplicate keys
multimap<int, int> mm;
mm.insert({key, val});
mm.equal_range(key);        // range of entries with same key

```

## 10. unordered\_set / set

```

#include <unordered_set>    // hash set – O(1) avg
#include <set>               // BST set – O(log n), sorted

unordered_set<int> us;
set<int> s;

// Common ops (same interface)
s.insert(x);
s.erase(x);
s.count(x);           // 1 or 0
s.find(x);            // iterator
s.size(), s.empty(), s.clear();

// set-only (ordered) ops
s.lower_bound(x);     // first element  $\geq$  x
s.upper_bound(x);     // first element > x
*s.begin();           // min
*prev(s.end());       // max
*s.rbegin();          // max (reverse)

// multiset – allows duplicates
multiset<int> ms;
ms.insert(x);
ms.erase(ms.find(x)); // erase ONE occurrence
ms.erase(x);          // erases ALL occurrences
ms.count(x);           // number of occurrences
ms.lower_bound(x);

```

## 11. Binary Search — `<algorithm>`

```

#include <algorithm>
// Array/vector must be sorted

// Check existence
binary_search(v.begin(), v.end(), x); // true/false

// Lower bound – first position where arr[i]  $\geq$  x
auto it = lower_bound(v.begin(), v.end(), x);
int idx = it - v.begin();

// Upper bound – first position where arr[i] > x
auto it = upper_bound(v.begin(), v.end(), x);

// Count occurrences
int cnt = upper_bound(all(v), x) - lower_bound(all(v), x);

// Manual binary search

```



```

int lo = 0, hi = n - 1;
while (lo ≤ hi) {
    int mid = lo + (hi - lo) / 2;    // avoids overflow (not (lo+hi)/2)
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}

// Search on answer (binary search on monotonic predicate)
auto feasible = [&](int x) → bool { ... };
int lo = 0, hi = max_val;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (feasible(mid)) hi = mid;
    else lo = mid + 1;
}
// lo is the answer

```

## 12. Linked List (Manual)

```

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

// Build: 1 → 2 → 3
ListNode* head = new ListNode(1);
head→next = new ListNode(2);
head→next→next = new ListNode(3);

// Traverse
for (ListNode* cur = head; cur; cur = cur→next)
    cout << cur→val;

// Reverse
ListNode* reverse(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* cur = head;
    while (cur) {
        ListNode* nxt = cur→next;
        cur→next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev;
}

// Fast & Slow pointer (midpoint / cycle)
ListNode* slow = head, *fast = head;
while (fast && fast→next) {
    slow = slow→next;
    fast = fast→next→next;
}
// slow is at midpoint

// Free memory
while (head) {
    ListNode* tmp = head;
    head = head→next;
    delete tmp;
}

```

## 13. Tree (Binary Tree)

```
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

// DFS traversals (recursive)
void inorder(TreeNode* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->val;
    inorder(root->right);
}

void preorder(TreeNode* root) {
    if (!root) return;
    cout << root->val;
    preorder(root->left);
    preorder(root->right);
}

void postorder(TreeNode* root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->val;
}

// BFS (level order)
void bfs(TreeNode* root) {
    if (!root) return;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        TreeNode* node = q.front(); q.pop();
        cout << node->val;
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
}

// Height
int height(TreeNode* root) {
    if (!root) return 0;
    return 1 + max(height(root->left), height(root->right));
}
```

## 14. Graph Representations

```
int n; // number of nodes

// Adjacency list (most common)
vector<vector<int>> graph(n);
graph[u].push_back(v); // directed
graph[u].push_back(v); graph[v].push_back(u); // undirected

// Weighted adjacency list
vector<vector<pair<int,int>>> graph(n); // {neighbor, weight}
graph[u].push_back({v, w});

// Adjacency matrix (dense graphs, n ≤ 1000)
int mat[1001][1001] = {};
```



```

mat[u][v] = 1;

// Edge list (for Kruskal's etc.)
vector<tuple<int,int,int>> edges;           // {weight, u, v}
edges.push_back({w, u, v});

// DFS
void dfs(int node, vector<bool>& vis, vector<vector<int>>& g) {
    vis[node] = true;
    for (int nei : g[node])
        if (!vis[nei]) dfs(nei, vis, g);
}

// DFS iterative
void dfs_iter(int start, vector<vector<int>>& g) {
    vector<bool> vis(n, false);
    stack<int> st;
    st.push(start);
    while (!st.empty()) {
        int node = st.top(); st.pop();
        if (vis[node]) continue;
        vis[node] = true;
        for (int nei : g[node]) st.push(nei);
    }
}

```

## 15. Union-Find (DSU)

```

struct DSU {
    vector<int> parent, rank;
    DSU(int n) : parent(n), rank(n, 0) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]); // path compression
        return parent[x];
    }
    bool unite(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) swap(px, py);
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
    bool connected(int x, int y) { return find(x) == find(y); }
};

// Usage
DSU dsu(n);
dsu.unite(0, 1);
dsu.connected(0, 2); // false

```

## 16. Segment Tree

```

struct SegTree {
    int n;
    vector<int> tree;
    SegTree(vector<int>& arr) : n(arr.size()), tree(4 * arr.size()) {
        build(arr, 0, 0, n - 1);
    }
    void build(vector<int>& arr, int node, int start, int end) {

```

```

    if (start == end) { tree[node] = arr[start]; return; }
    int mid = (start + end) / 2;
    build(arr, 2*node+1, start, mid);
    build(arr, 2*node+2, mid+1, end);
    tree[node] = tree[2*node+1] + tree[2*node+2]; // sum tree
}

void update(int node, int start, int end, int idx, int val) {
    if (start == end) { tree[node] = val; return; }
    int mid = (start + end) / 2;
    if (idx ≤ mid) update(2*node+1, start, mid, idx, val);
    else update(2*node+2, mid+1, end, idx, val);
    tree[node] = tree[2*node+1] + tree[2*node+2];
}

int query(int node, int start, int end, int l, int r) {
    if (r < start || end < l) return 0;
    if (l ≤ start && end ≤ r) return tree[node];
    int mid = (start + end) / 2;
    return query(2*node+1, start, mid, l, r) +
           query(2*node+2, mid+1, end, l, r);
}

// Public interface wrappers
void update(int idx, int val) { update(0, 0, n-1, idx, val); }
int query(int l, int r) { return query(0, 0, n-1, l, r); }
};

```

## 17. Trie (Prefix Tree)

```

struct TrieNode {
    TrieNode* children[26] = {};
    bool end = false;
};

struct Trie {
    TrieNode* root = new TrieNode();

    void insert(const string& word) {
        TrieNode* cur = root;
        for (char c : word) {
            int i = c - 'a';
            if (!cur->children[i])
                cur->children[i] = new TrieNode();
            cur = cur->children[i];
        }
        cur->end = true;
    }

    bool search(const string& word) {
        TrieNode* cur = root;
        for (char c : word) {
            int i = c - 'a';
            if (!cur->children[i]) return false;
            cur = cur->children[i];
        }
        return cur->end;
    }

    bool startsWith(const string& prefix) {
        TrieNode* cur = root;
        for (char c : prefix) {
            int i = c - 'a';
            if (!cur->children[i]) return false;
            cur = cur->children[i];
        }
        return true;
    }
};

```



```
}  
};
```

## 18. Useful `<algorithm>` Functions

```
#include <algorithm>  
  
sort(v.begin(), v.end());  
sort(v.begin(), v.end(), greater<int>());  
sort(v.begin(), v.end(), [](int a, int b){ return a > b; });  
  
// Stable sort (preserves relative order of equal elements)  
stable_sort(v.begin(), v.end());  
  
reverse(v.begin(), v.end());  
rotate(v.begin(), v.begin()+k, v.end()); // rotate left by k  
  
// Min / Max  
min(a, b), max(a, b);  
min({a, b, c}), max({a, b, c}); // variadic  
*min_element(all(v)), *max_element(all(v));  
auto [mn, mx] = minmax_element(all(v));  
  
// Permutations  
next_permutation(all(v)); // in-place next permutation  
prev_permutation(all(v));  
// Loop all permutations:  
sort(all(v));  
do { /* use v */ } while (next_permutation(all(v)));  
  
// Partitioning  
partition(all(v), [](int x){ return x % 2 == 0; });  
nth_element(all(v), v.begin()+k, v.end()); // O(n) partial sort, v[k] is correct  
  
// Fill / copy  
fill(all(v), 0);  
copy(src.begin(), src.end(), dst.begin());  
  
// Set operations (on sorted ranges)  
set_union(all(a), all(b), back_inserter(result));  
set_intersection(all(a), all(b), back_inserter(result));  
set_difference(all(a), all(b), back_inserter(result));  
  
// Count / find  
count(all(v), x);  
count_if(all(v), [](int x){ return x > 5; });  
find(all(v), x);  
find_if(all(v), [](int x){ return x > 5; });  
any_of(all(v), [](int x){ return x > 0; });  
all_of(all(v), [](int x){ return x > 0; });  
none_of(all(v), [](int x){ return x < 0; });  
  
// Numeric  
#include <numeric>  
accumulate(all(v), 0); // sum  
accumulate(all(v), 1, multiplies<int>()); // product  
partial_sum(all(v), out.begin()); // prefix sum  
iota(v.begin(), v.end(), 0); // fill 0,1,2,...,n-1  
gcd(a, b), lcm(a, b); // C++17
```

## 19. Math & Bit Tricks



```

#include <cmath>

abs(x)                // int abs
fabs(x)               // float abs
sqrt(x), cbrt(x)
pow(x, y)             // float – use custom for int
ceil(x), floor(x), round(x)
log(x), log2(x), log10(x)
__gcd(a, b)           // gcd (older, still common in CP)

// Modular arithmetic
const int MOD = 1e9 + 7;
(a + b) % MOD
(a * b) % MOD
((a - b) % MOD + MOD) % MOD // handles negative mod

// Fast power (modular exponentiation)
long long power(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) result = result * base % mod;
        base = base * base % mod;
        exp >>= 1;
    }
    return result;
}

// Bit manipulation
x & 1                // check odd/even
x >> 1              // divide by 2
x << 1              // multiply by 2
x & (x - 1)         // clear lowest set bit
x & (-x)            // isolate lowest set bit
__builtin_popcount(x) // count set bits (int)
__builtin_popcountll(x) // count set bits (long long)
__builtin_clz(x)      // count leading zeros
__builtin_ctz(x)      // count trailing zeros
__builtin_parity(x)   // parity of set bits

// Check power of 2
bool isPow2 = x > 0 && !(x & (x - 1));

// Iterate over all subsets of mask
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) { }

```

## 20. Pairs & Tuples

```

#include <utility>
#include <tuple>

pair<int,int> p = {1, 2};
p.first, p.second;
make_pair(1, 2);

// Structured bindings (C++17)
auto [a, b] = p;

// Sorting vector of pairs (sorts by first, then second)
vector<pair<int,int>> v;
sort(all(v));
sort(all(v), [](auto& a, auto& b){ return a.second < b.second; });

// Tuple
tuple<int, int, int> t = {1, 2, 3};

```

```

get<0>(t), get<1>(t), get<2>(t);
auto [x, y, z] = t; // C++17 structured binding
make_tuple(1, 2, 3);
tie(a, b, c) = t; // unpack into existing variables

```

## 21. Input / Output (Competitive Style)

```

#include <bits/stdc++.h>
using namespace std;

// Fast I/O – always add at top of main
ios_base::sync_with_stdio(false);
cin.tie(NULL);

// Read
int n; cin >> n;
int a, b; cin >> a >> b;
string s; cin >> s; // reads word
getline(cin, s); // reads full line (use after cin >> if needed)

// Read array
int arr[n];
for (int i = 0; i < n; i++) cin >> arr[i];
vector<int> v(n);
for (auto& x : v) cin >> x;

// Read until EOF
int x;
while (cin >> x) v.push_back(x);

// Output
cout << x << "\n"; // use "\n" not endl (endl flushes buffer)
printf("%.2f\n", x); // formatted output (faster than cout for floats)
cout << fixed << setprecision(2) << x; // cout equivalent

```

## 22. Common Patterns & Gotchas

```

// Integer overflow
int a = 1e9, b = 1e9;
long long c = (long long)a * b; // cast before multiply!
long long c = 1LL * a * b; // 1LL forces long long arithmetic

// Array initialization
int dp[1001][1001];
memset(dp, 0, sizeof(dp)); // fill with 0
memset(dp, -1, sizeof(dp)); // fill with -1
memset(dp, 0x3f, sizeof(dp)); // fill with ~1e9 (useful "infinity")

// Prefix sum
vector<int> pre(n + 1, 0);
for (int i = 0; i < n; i++) pre[i+1] = pre[i] + arr[i];
// range sum [l, r] (0-indexed, inclusive)
int sum = pre[r+1] - pre[l];

// Coordinate compression
vector<int> vals = arr;
sort(all(vals));
vals.erase(unique(all(vals)), vals.end());
auto compress = [&](int x) {
    return lower_bound(all(vals), x) - vals.begin();
};

// Lambda with capture

```

```

auto fn = [&](int x) { return x * x; };
sort(all(v), [&](int a, int b){ return a > b; });

// Recursive lambda (C++14+)
function<int(int)> fib = [&](int n) → int {
    if (n ≤ 1) return n;
    return fib(n-1) + fib(n-2);
};

// Multiple return values
pair<int,int> solve() { return {a, b}; }
auto [x, y] = solve();

// Max/min of different signed/unsigned (avoids warning)
(int)v.size() - 1 // cast size_t to int
min((int)v.size(), k)

// Topological sort (Kahn's algorithm)
vector<int> indegree(n, 0);
for (auto& [u, v] : edges) indegree[v]++;
queue<int> q;
for (int i = 0; i < n; i++) if (!indegree[i]) q.push(i);
vector<int> topo;
while (!q.empty()) {
    int u = q.front(); q.pop();
    topo.push_back(u);
    for (int v : graph[u])
        if (--indegree[v] == 0) q.push(v);
}

```

## 23. STL Container Complexity Summary

| Container      | Access       | Search      | Insert           | Delete           | Ordered? |
|----------------|--------------|-------------|------------------|------------------|----------|
| vector         | $O(1)$       | $O(n)$      | $O(1) * / O(n)$  | $O(1) * / O(n)$  | No       |
| array          | $O(1)$       | $O(n)$      | —                | —                | No       |
| deque          | $O(1)$       | $O(n)$      | $O(1)$ both ends | $O(1)$ both ends | No       |
| stack          | $O(1)$ top   | —           | $O(1)$           | $O(1)$           | No       |
| queue          | $O(1)$ front | —           | $O(1)$           | $O(1)$           | No       |
| priority_queue | $O(1)$ top   | —           | $O(\log n)$      | $O(\log n)$      | Heap     |
| unordered_map  | $O(1)$ avg   | $O(1)$ avg  | $O(1)$ avg       | $O(1)$ avg       | No       |
| map            | $O(\log n)$  | $O(\log n)$ | $O(\log n)$      | $O(\log n)$      | Yes      |
| unordered_set  | —            | $O(1)$ avg  | $O(1)$ avg       | $O(1)$ avg       | No       |
| set            | —            | $O(\log n)$ | $O(\log n)$      | $O(\log n)$      | Yes      |
| multiset       | —            | $O(\log n)$ | $O(\log n)$      | $O(\log n)$      | Yes      |

vector *push\_back/pop\_back* is  $O(1)$  amortized. *Insert/erase at middle* is  $O(n)$ .